

# Week 5 — Agent Architecture with LLM Tool Use

**Author:** Arnav Mahale (arnav.mahale@duke.edu) **Course:** AIPI 561 — Operationalizing AI

---

## 1. Overview

This week I built an AI agent that answers TechCorp business questions by pairing an LLM (Google Gemini) with three local tools: a SQLite employee lookup, a policy document search, and an expense-limit lookup. The LLM does the *reasoning* (which tool to call, with what arguments) and *synthesis* (turning raw tool output into a human answer); my code does the *execution* (running the tool against real data) and the *bookkeeping* (token and cost tracking).

All code is in `app_starter.py`. The 10-query test transcript is in `test_results.md`, reproduced in §6.

---

## 2. Model choice (important note)

The README specifies `gemini-2.5-pro`. In practice, Google now sets the **free-tier quota for gemini-2.5-pro to zero** — every request returns `429 RESOURCE_EXHAUSTED ... limit: 0`, i.e. Pro requires a billing-enabled account. Per the instructor’s announcement (June 10) that we may “switch to another standard model of your choice,” I run the agent on `gemini-2.5-flash-lite` (with `gemini-2.5-flash` as an alternate). Both are standard Gemini models on the free tier. The model is configurable via the `GEMINI_MODEL` environment variable:

```
MODEL = os.getenv("GEMINI_MODEL", "gemini-2.5-flash")
```

I also learned the free tier caps usage at roughly **20 requests/day per model**. Because each query makes two LLM calls (route + synthesize), a full 10-query run is ~20 calls — right at the cap. The quota is *per model*, so switching models yields a fresh bucket. This is itself a nice operational lesson for the course’s “real constraints: cost” theme.

---

## 3. Code structure

### 3.1 Tools

Every tool subclasses a tiny `Tool` base class (name, description, `execute()`), so the agent can treat them uniformly.

| Tool                            | Data source                                           | What <code>execute()</code> does                                                                                                                                                                                                                                                                                |
|---------------------------------|-------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>EmployeeLookupTool</code> | <code>data/techcorp.db</code> (SQLite, 10k employees) | <code>SELECT * FROM employees WHERE id = ?</code> (exact) or <code>WHERE name LIKE ?</code> (partial). Returns matching rows as JSON, or <code>"Employee not found"</code> . Uses <code>sqlite3.Row</code> so results are keyed by column name.                                                                 |
| <code>PolicySearchTool</code>   | <code>data/documents.json</code> (74 docs)            | Scores each document by how many query <i>words</i> appear in its title (weight 2) and content (weight 1), returns the top-N with a 500-char snippet. Word-level scoring means “travel policy” still matches the “ <i>Travel and Expense Policy</i> ” doc even though that exact phrase never appears verbatim. |

| Tool             | Data source                     | What <code>execute()</code> does                                                                                                                                        |
|------------------|---------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ExpenseQueryTool | <code>data/policies.json</code> | Looks up <code>policies["expense"]["approval_limits"][role]</code> , returns "Approval limit for {role}: \${amount}" or a "Role not found" message listing valid roles. |

Documents and policies are loaded **once in `__init__`**, not on every call.

### 3.2 Agent reasoning loop

`Agent.query()` implements a two-call route → execute → synthesize loop:

1. **Build the system prompt** (`_build_system_prompt`) — describes the agent's purpose, lists each tool, gives argument hints, and defines a strict reply format (`TOOL: <name> / ARGS: key=value`). The user's role is injected as context (not yet enforced — see §5).
2. **Route (LLM call #1)** — Gemini reads the system prompt + question and either emits a `TOOL:` block or answers directly.
3. **Parse** (`_parse_tool_call`) — regex extracts the tool name and a comma-separated `key=value` argument list. If there's no `TOOL:` line, the model's direct reply is used as the answer (no second call — saves quota).
4. **Execute** — the named tool runs locally and returns real data.
5. **Synthesize (LLM call #2)** — the tool result is handed back to Gemini with an instruction to answer *using only that data* (this is what keeps it from hallucinating; see queries 8 and 10, where it correctly says the info isn't in the documents).
6. **Track** tokens and cost across *both* calls.

A small `_generate()` wrapper retries on transient 429 rate-limit errors with exponential backoff, so a brief per-minute spike doesn't kill a run (this fired harmlessly on query 6 below).

### 3.3 Cost tracking

`usage_metadata` from each Gemini response gives `prompt_token_count` and `candidates_token_count`. These are summed across both calls per query and run through the assignment's rate card:

```
def _estimate_query_cost(self, input_tokens, output_tokens):
    input_cost = (input_tokens / 1_000_000) * 0.075    # $/1M input tokens
    output_cost = (output_tokens / 1_000_000) * 0.300   # $/1M output tokens
    return input_cost + output_cost
```

`get_metrics()` returns running totals: `total_queries`, `total_tokens`, `total_cost`, and `avg_cost_per_query` (guarded against divide-by-zero).

Note: I kept the README's documented Gemini 2.5 **Pro** rate card for the cost-tracking exercise, even though execution runs on Flash-lite, so the cost *formula* matches the assignment. The mechanism (token capture → per-token rate → running totals) is identical regardless of the rate constants.

## 4. How to run

```
cd week5
pip install -r requirements.txt          # google-genai, python-dotenv
echo 'GOOGLE_API_KEY=YOUR_KEY' > .env   # free key from aistudio.google.com/app/apikey
python3 app_starter.py                  # built-in 3-query smoke test
python3 run_tests.py                    # full 10-query transcript -> test_results.md
```

GOOGLE\_API\_KEY is read from week5/.env (gitignored, never committed).

---

## 5. Access control (Week 6 preview)

`Agent.query()` accepts a `user_role` and threads it into the system prompt, but it is **not enforced** this week — every query currently returns full rows including sensitive fields (e.g. `salary`, `ssn`). `data/access_control.json` defines which roles may see which fields; Week 6 will use it to redact/deny. The hook is already in place so that enforcement is an additive change, not a rewrite.

---

## 6. Test results — 10 queries

Run on `gemini-2.5-flash-lite`. Full transcript in `test_results.md`.

| #  | Role     | Query                                 | Tool used                    | Result (summary)                              |
|----|----------|---------------------------------------|------------------------------|-----------------------------------------------|
| 1  | engineer | What is the travel policy?            | <code>policy_search</code>   | Pre-approval + per-level domestic/intl limits |
| 2  | engineer | Expense approval limit for a manager? | <code>expense_query</code>   | \$5,000                                       |
| 3  | finance  | Expense approval limit for a VP?      | <code>expense_query</code>   | \$100,000                                     |
| 4  | finance  | How much can a director approve?      | <code>expense_query</code>   | \$25,000                                      |
| 5  | hr       | Look up employee Brian Yang           | <code>employee_lookup</code> | VP Engineering, Engineering dept              |
| 6  | hr       | Find employee with ID 1               | <code>employee_lookup</code> | Brian Yang (429 retry recovered)              |
| 7  | engineer | Policy on remote work?                | <code>policy_search</code>   | Eligibility + Full-Remote/Hybrid rules        |
| 8  | employee | Parental leave / benefits policy?     | <code>policy_search</code>   | Correctly: not in the documents               |
| 9  | manager  | Expense approval limit for an ic3?    | <code>expense_query</code>   | \$2,000                                       |
| 10 | engineer | Security policy for handling data?    | <code>policy_search</code>   | Correctly grounds answer in GDPR doc          |

### Cost summary

| Metric           | Value             |
|------------------|-------------------|
| Total queries    | <b>10</b>         |
| Total tokens     | <b>6,516</b>      |
| Total cost       | <b>\$0.000615</b> |
| Avg cost / query | <b>\$0.000061</b> |

At ~\$0.00006 per query, ~16,000 queries would cost about \$1 on this rate card — useful context for the cost/scale discussions in the weeks ahead.

### Observations

- **Grounding works.** Queries 8 and 10 had no exact matching policy, and the agent said so instead of inventing one (query 10 helpfully surfaced the related GDPR doc). This is a direct result of the synthesis prompt constraining the model to the tool output.
- **Routing is reliable.** All 10 queries selected the correct tool and passed sensible arguments (e.g. `role=manager`, `employee_id=1`) with no manual nudging.
- **Cost is dominated by the policy-search queries** (1, 7, 10), because returning 500-char document snippets to the synthesis call uses the most tokens (~1,100 vs ~320 for a one-line expense lookup).